

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бекэнд разработка

Отчет

**Домашняя работа №4
«Технический дизайн микросервисной архитектуры»**

Выполнил:
Дементьев Антон
K3341

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Вариант

Сайт для поиска работы

- Вход
- Регистрация
- Личный кабинет пользователя (с резюме)
- Поиск вакансий с фильтрацией по отрасли, зарплате, опыту
- Страница с деталями вакансии (описание, требования, компания)
- Личный кабинет работодателя (управление вакансиями)

Задача

- 1) Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
- 2) Спроектировать микросервисную архитектуру: отразить взаимосвязь между микросервисами, описать способы их взаимодействия друг с другом
- 3) Спроектировать разделение БД на обособленные БД
- 4) Спроектировать и описать эндпоинты, необходимые для обеспечения межсервисного взаимодействия, в формате спецификации OpenAPI
- 5) Описать варианты запросов и ответов, задокументировать возможные ошибки
- 6) Сформировать документ с конкретными требованиями и шагами для разделения монолитного бэкенд-приложения на отдельные микросервисы

Ход работы

В работе монолитное backend-приложение сайта поиска работы (ЛР 1) разделено на семь микросервисов и API Gateway по принципу database-per-service. Спроектированы архитектура и способы взаимодействия сервисов, разделение базы данных, межсервисное API в формате OpenAPI 3.0 (файл openapi-internal.yaml), ошибки и деградация при сбоях, требования и пошаговый план перехода. Публичное API для клиентов (openapi.yaml из ДЗ 2) не меняется: клиент по-прежнему видит те же 50 операций.

1. Исходное состояние и принципы разбиения

1.1. Что есть в монолите

Монолит (ЛР 1) состоит из 10 контроллеров и 50 операций, работает с одной базой PostgreSQL из 15 таблиц и 19 внешних ключей. Данные разных областей связаны напрямую, поэтому при разделении нужно заменить следующие места:

Место связности в монолите	Как реализовано сейчас	Что с этим делать
Отклик на вакансию (№40)	Один процесс читает vacancies и resumes и пишет в applications	Application Service запрашивает вакансию и резюме у их владельцев
Списки и карточка откликов (№39, 41, 42)	JOIN applications, vacancies, companies, industries, resumes, user_profiles	Сборка ответа из пакетных запросов к Vacancy, Resume и Identity
Просмотр резюме работодателем (№17)	JOIN applications, vacancies, companies	Вызов Application Service: access-check
Удаление вакансии, резюме, компании (№19, 31, 37)	ON DELETE RESTRICT: запрет обеспечивает БД	Проверка через API (exists) вместо внешнего ключа
Ссылки на отрасли и навыки	Внешние ключи на industries и skills из 5 таблиц	Логические ссылки и проверка через Reference Service
Рекомендации (№48, 49)	Читаются vacancies, vacancy_skills, resumes, resume_skills, resume_experiences	Собственный индекс Recommendation Service, поддерживаемый событиями
Коды подтверждения	Письмо «отправляется» прямо в запросе	Асинхронная отправка через Notification Service
Проверка JWT, лимит запросов	Middleware внутри монолита	Переезжает в API Gateway

1.2. Принципы разбиения

- Границы сервисов проходят по областям бизнеса (аккаунты, справочники, вакансии, резюме, отклики, подбор, уведомления): у каждой области один владелец данных.
- Database-per-service: у каждого сервиса собственная БД, чужие таблицы недоступны ни на чтение, ни на запись; обмен данными только через API или события.
- Публичный контракт не меняется: все 50 операций, коды ответов и ошибки из ДЗ 2 остаются прежними, поэтому клиенты и тесты (Postman из ДЗ 3) продолжают работать.
- Синхронные вызовы используются там, где нужен немедленный ответ или проверка. Асинхронные события используются там, где достаточно согласованности «в конечном итоге» (индекс рекомендаций, письма).
- Распределённых транзакций нет. Целостность между сервисами обеспечивается проверками, порядком операций и сверкой (раздел 4.5).

2. Декомпозиция на микросервисы

2.1. Состав сервисов

Сервис (порт)	Ответственность	Публичные операции (№ из ДЗ 2)	Таблицы монолита	БД
API Gateway (8080)	Единая точка входа: маршрутизация, проверка JWT, лимит запросов, X-Request-Id, сбор состояния	№50	нет	нет (Redis для счётчиков)
Identity (8001)	Регистрация, вход, токены, подтверждение email, профиль; выпуск сервисных токенов, JWKS	№1–11	users, user_profiles, refresh_tokens, verification_tokens	identity_db
Reference (8002)	Справочники отраслей и навыков	№12–14	industries, skills	reference_db
Vacancy (8003)	Компании, вакансии и их навыки, поиск вакансий, избранное	№27–38, №45–47	companies, vacancies, vacancy_skills, favorite_vacancies	vacancy_db
Resume (8004)	Резюме, опыт, образование, навыки резюме	№15–26	resumes, resume_experiences, resume_educations, resume_skills	resume_db
Application (8005)	Отклики и их статусы, отклики на вакансию	№39–44	applications	application_db
Recommendation (8006)	Подбор вакансий для резюме и кандидатов для вакансии	№48, 49	нет (новый индекс)	recommendation_db
Notification (8007)	Отправка писем с кодами подтверждения	нет (только тестовый ящик)	нет (новые таблицы)	notification_db

Итого 50 операций распределены так: Identity 11, Vacancy 15, Resume 12, Application 6, Reference 3, Recommendation 2, шлюз 1.

2.2. Обоснование границ

- Reference выделен отдельно: справочники нужны всем сервисам, меняются редко и хорошо кешируются, а владелец у них один. Это же убирает пять внешних ключей на industries и skills.
- Избранное остаётся в Vacancy: запись избранного удаляется вместе с вакансией (CASCADE), уникальна по паре (user_id, vacancy_id), а выдача содержит карточки вакансий. В одном сервисе всё это делается одним запросом к своей БД, без сети.
- Компании и вакансии объединены: вакансия не существует без компании (RESTRICT), их правила и поиск неразрывны.
- Application выделен, так как у откликов собственный жизненный цикл (статусы, отзыв). Он не владеет вакансиями и резюме, а только ссылается на них, поэтому хранит идентификаторы без внешних ключей.
- Recommendation выделен из-за особой нагрузки: он читает много данных и считает, а его модель данных (индекс) отличается от боевой. Он масштабируется независимо и его сбой не влияет на остальные функции.

- Notification выделен, потому что отправка письма это побочный эффект: она не должна задерживать ответ клиенту, может повторяться и зависит от внешнего почтового сервиса.
- Профиль пользователя остаётся в Identity: он создаётся вместе с аккаунтом (одна транзакция) и связан с ним 1:1, разделять их не имеет смысла.

2.3. API Gateway и маршрутизация

Шлюз принимает все запросы клиентов, проверяет подпись JWT, применяет лимит запросов (счётчики в Redis, вместо памяти процесса), добавляет заголовок X-Request-Id и направляет запрос сервису-владельцу. Правила применяются сверху вниз, более конкретные стоят выше общих (иначе пути вида `/vacancies/{id}/applications` попали бы в Vacancy Service):

№	Шаблон пути (после <code>/api/v1</code>)	Сервис	Операции
1	<code>/vacancies/{id}/applications</code>	Application	№39, 40
2	<code>/vacancies/{id}/recommended-resumes</code>	Recommendation	№49
3	<code>/resumes/{id}/recommended-vacancies</code>	Recommendation	№48
4	<code>/applications, /applications/**</code>	Application	№41–44
5	<code>/auth/**, /users/**</code>	Identity	№1–11
6	<code>/industries, /skills</code>	Reference	№12–14
7	<code>/companies/**, /vacancies/**, /favorites/**</code>	Vacancy	№27–38, №45–47
8	<code>/resumes/**</code>	Resume	№15–26
9	<code>/health</code>	шлюз (опрашивает GET <code>/internal/v1/health</code> всех сервисов)	№50
10	<code>/dev/mailbox</code> (только тестовый стенд)	Notification: <code>/internal/v1/dev/mailbox</code>	автотесты
11	<code>/internal/**</code>	не маршрутизируется, ответ 404	нет

3. Архитектура и взаимодействие сервисов

3.1. Компоненты

Схема на рисунке 1 показывает клиентов, шлюз, семь сервисов с собственными БД и брокер сообщений. Каждый сервис запускается отдельным контейнером на том же стеке, что и монолит (Node.js, TypeScript, Express, routing-controllers, TypeORM); БД каждого сервиса это отдельная база PostgreSQL с отдельным пользователем, у которого нет прав на другие базы. Для событий используется RabbitMQ (topic exchange `jobsearch.events`), для счётчиков лимитов шлюза Redis. Редактируемая версия схем приложена файлом `microservices.drawio` (три страницы).

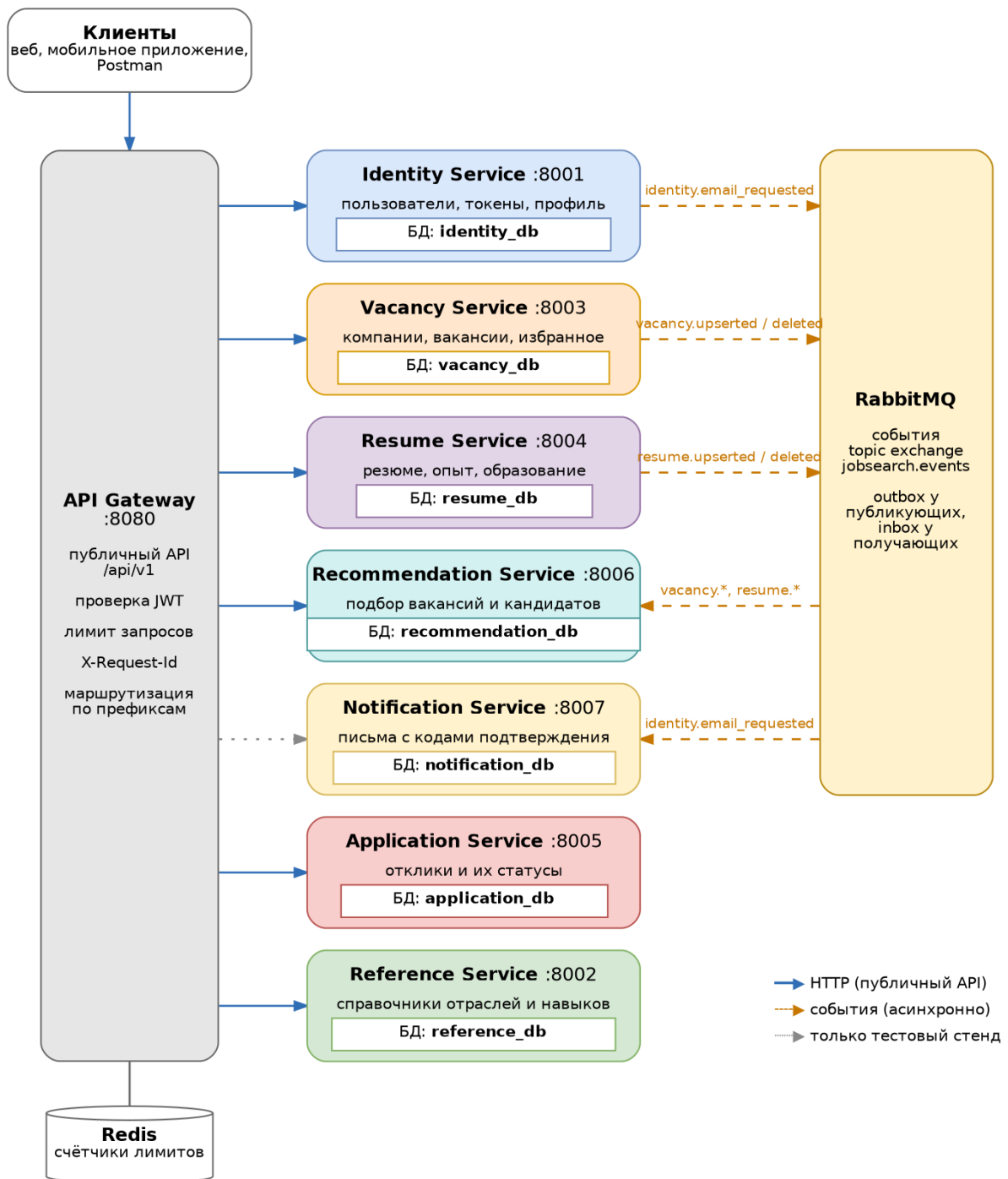


Рисунок 1 — Компоненты микросервисной архитектуры

3.2. Способы взаимодействия

Способ	Когда применяется	Технология и гарантии
Синхронный запрос-ответ	Нужен ответ прямо сейчас: проверка существования, права, данные для карточки, запрет удаления	HTTP/REST, JSON, внутреннее API /internal/v1. Тайм-аут 2 с (пакетные 3 с), не более 2 повторов только для GET и lookup, circuit breaker
Асинхронные события	Достаточно согласованности в конечном итоге: обновление индекса рекомендаций, письма	RabbitMQ. Доставка «как минимум один раз», публикация через outbox, потребители идемпотентны (inbox)
Кеш справочников	Названия и проверка существования отраслей и навыков	В памяти сервиса, TTL 10 минут; при недоступности Reference используются устаревшие значения
Выгрузка данных	Первичная загрузка и перестроение индекса рекомендаций	GET /internal/v1/vacancies/export и /resumes/export, курсорная пагинация

3.3. Синхронные вызовы

Рисунок 2 показывает, какие сервисы вызывают друг друга напрямую. Номера стрелок соответствуют таблице ниже.

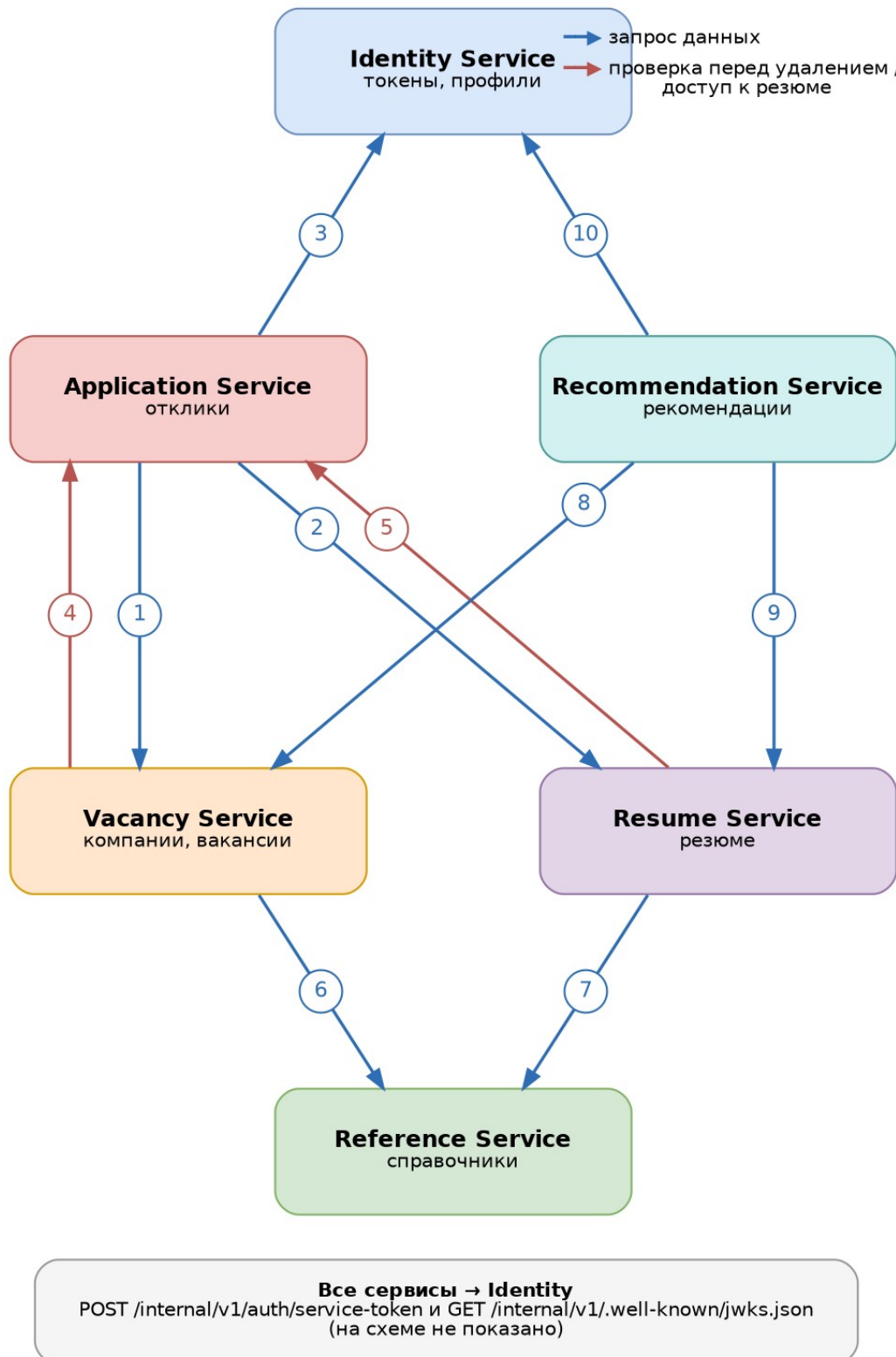


Рисунок 2 — Синхронные вызовы между сервисами

№	Откуда → куда	Эндпоинты	Зачем	Если сервис недоступен
1	Application → Vacancy	GET /vacancies/{id}, POST /vacancies/lookup	Проверка вакансии при отклике; карточки в списках откликов	Критично: 503
2	Application → Resume	GET /resumes/{id}, POST /resumes/lookup	Проверка резюме при отклике; карточки резюме	Критично: 503
3	Application → Identity	POST /users/lookup	Имена кандидатов в списках, контакты в карточке отклика	Имена: пустые поля; контакты: 503
4	Vacancy → Application	GET /applications/exists? vacancy_id	Запрет удаления вакансии с откликами	Удаление не выполняется: 503
5	Resume → Application	GET /applications/exists? resume_id, GET /applications/access-check	Запрет удаления резюме с откликами; доступ работодателя к резюме	Критично: 503 (доступ не даётся)
6	Vacancy → Reference	POST /industries/lookup, POST /skills/lookup	Проверка ссылок при записи; названия в карточках	Чтение: устаревший кеш; запись без кеша: 503
7	Resume → Reference	POST /industries/lookup, POST /skills/lookup	То же для резюме	То же
8	Recommendation → Vacancy	GET /vacancies/{id}, POST /vacancies/lookup, GET /vacancies/export	Вакансия и её владелец для подбора кандидатов; карточки в ответе; загрузка индекса	Критично: 503
9	Recommendation → Resume	GET /resumes/{id}, POST /resumes/lookup, GET /resumes/export	Резюме и его владелец для подбора вакансий; карточки; загрузка индекса	Критично: 503
10	Recommendation → Identity	POST /users/lookup	Имена кандидатов в рекомендациях	Имена: пустые поля

Кроме перечисленного, каждый сервис получает сервисный токен (POST /auth/service-token) и ключи проверки JWT (GET /.well-known/jwks.json) у Identity; на схеме эти связи не показаны. Вызовы не вложены: Application вызывает Vacancy, а Vacancy при обработке этого запроса не обращается обратно в Application. Поэтому взаимные зависимости Application, Vacancy и Resume не приводят к взаимной блокировке.

3.4. События

Событие	Публикует	Получает	Когда возникает	Что внутри
identity.email_request_id	Identity	Notification	Регистрация (№1), повторная отправка (№3), запрос смены пароля (№8)	user_id, to_email, template, code, ttl_minutes
vacancy.upserted	Vacancy	Recommendation	Создание и изменение вакансии, смена навыков, закрытие и открытие (№34, 36, 38)	Снимок для индекса: id, отрасль, зарплата, опыт, is_active, навыки, updated_at
vacancy.deleted	Vacancy	Recommendation	Удаление вакансии (№37)	vacancy_id, deleted_at
resume.upserted	Resume	Recommendation	Создание, изменение, публикация резюме, изменение опыта и навыков (№16, 18, 20–22, 26)	Снимок для индекса: id, владелец, отрасль, зарплата, is_published, периоды работы, навыки
resume.deleted	Resume	Recommendation	Удаление резюме (№19)	resume_id, deleted_at

Гарантии доставки. Событие записывается в таблицу outbox_events в той же транзакции, что и изменение данных, а отдельный процесс публикует его в RabbitMQ и помечает опубликованным (так изменение и событие не расходятся). Потребитель хранит id обработанных событий в inbox_events и не выполняет событие повторно (доставка возможна больше одного раза). Порядок событий одной вакансии или резюме

обеспечивается ключом маршрутизации и полем `updated_at`: потребитель игнорирует событие, если в индексе уже более новая версия. После пяти неудачных попыток сообщение уходит в очередь ошибок (DLQ) для разбора. Формат сообщений приведён в разделе 5.5.

3.5. Безопасность

- Токены пользователей. Identity подписывает access-токены асимметричным ключом (RS256) вместо общего секрета HS256 из монолита; публичные ключи отдаются через JWKS. Шлюз проверяет подпись и срок, но и каждый сервис проверяет токен самостоятельно, не доверяя заголовкам шлюза.
- Токены сервисов. Для внутренних вызовов сервис получает у Identity токен по схеме `client credentials` (`POST /internal/v1/auth/service-token`). Токен живёт 5 минут и привязан к получателю (`aud`), поэтому токен для `vacancy-service` не подойдёт для `resume-service`. Контакты пользователей (`include_contacts`) выдаются только `application-service`.
- Сеть. Внутренние порты сервисов недоступны снаружи, шлюз не маршрутизирует `/internal/**`; БД доступна только своему сервису.
- Секреты (`client_secret`, пароли БД, приватный ключ) хранятся в переменных окружения или хранилище секретов, а не в коде; между компонентами используется TLS.
- Коды подтверждения. Код передаётся в событии `identity.email_requested` открытым текстом (иначе письмо не собрать): сообщение живёт недолго (TTL), содержимое не пишется в логи и в `email_log`, доступ к очереди ограничен, канал защищён TLS.
- Тестовый ящик `GET /dev/mailbox` включается только на тестовом стенде (`MAIL_DEBUG=true`) и отсутствует в продуктивной среде.

3.6. Отказоустойчивость

Механизм	Правило
Тайм-ауты	Соединение 500 мс; ответ 2 с, для пакетных запросов и выгрузок 3 с. Превышение: 504 UPSTREAM_TIMEOUT
Повторы	Не более двух, с нарастающей паузой и случайным разбросом; только для GET и lookup (они не меняют данные). POST с побочным эффектом сам не повторяется
Circuit breaker	5 ошибок за 10 секунд открывают цепь на 30 секунд; пока цепь открыта, вызывающий сервис сразу отвечает 503 SERVICE_UNAVAILABLE с Retry-After
Деградация	Некритичные данные (имена кандидатов, названия справочников) заменяются пустыми значениями или кешем; критичные (права, владелец, проверка удаления) ошибку не скрывают
Изоляция	Отдельные пулы соединений на каждую зависимость; отказ Recommendation или Notification не влияет на остальные функции
Готовность	GET /internal/v1/health у каждого сервиса проверяет БД и брокер; оркестратор и шлюз используют его для readiness и для публичного №50

3.7. Сквозные сценарии

Ниже показаны четыре сценария, в которых сервисы работают вместе. Остальные операции выполняются внутри одного сервиса.

Регистрация и подтверждение email (№1 и №2)

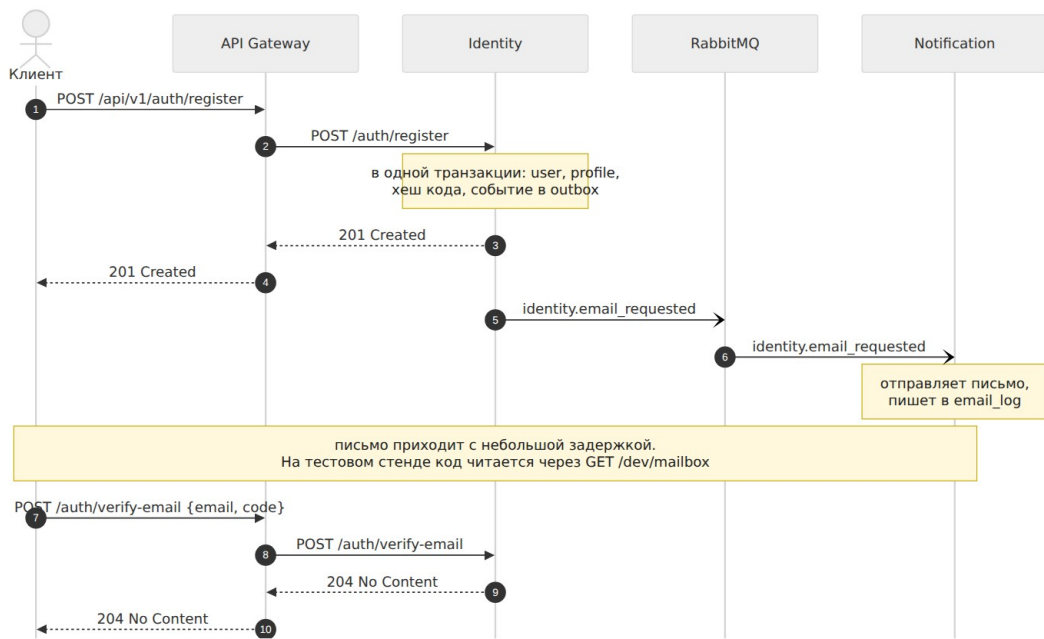


Рисунок 3 — Регистрация и подтверждение email

Отклик на вакансию (№40)

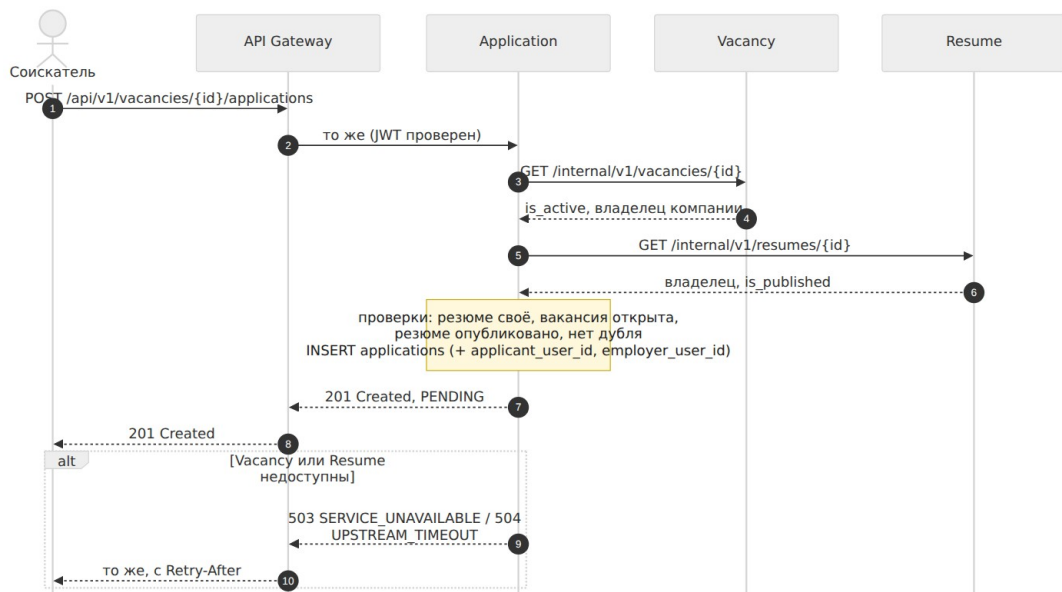


Рисунок 4 — Отклик на вакансию

Удаление вакансии (№37): «сначала закрыть, потом проверить»

В монолите удаление вакансии с откликами запрещало БД (RESTRICT). Теперь Vacancy Service сначала закрывает вакансию (новые отклики не принимаются), затем спрашивает Application Service, есть ли отклики, и лишь после этого удаляет или возвращает прежнее состояние. Так исчезает окно, в котором отклик мог бы появиться между проверкой и удалением.

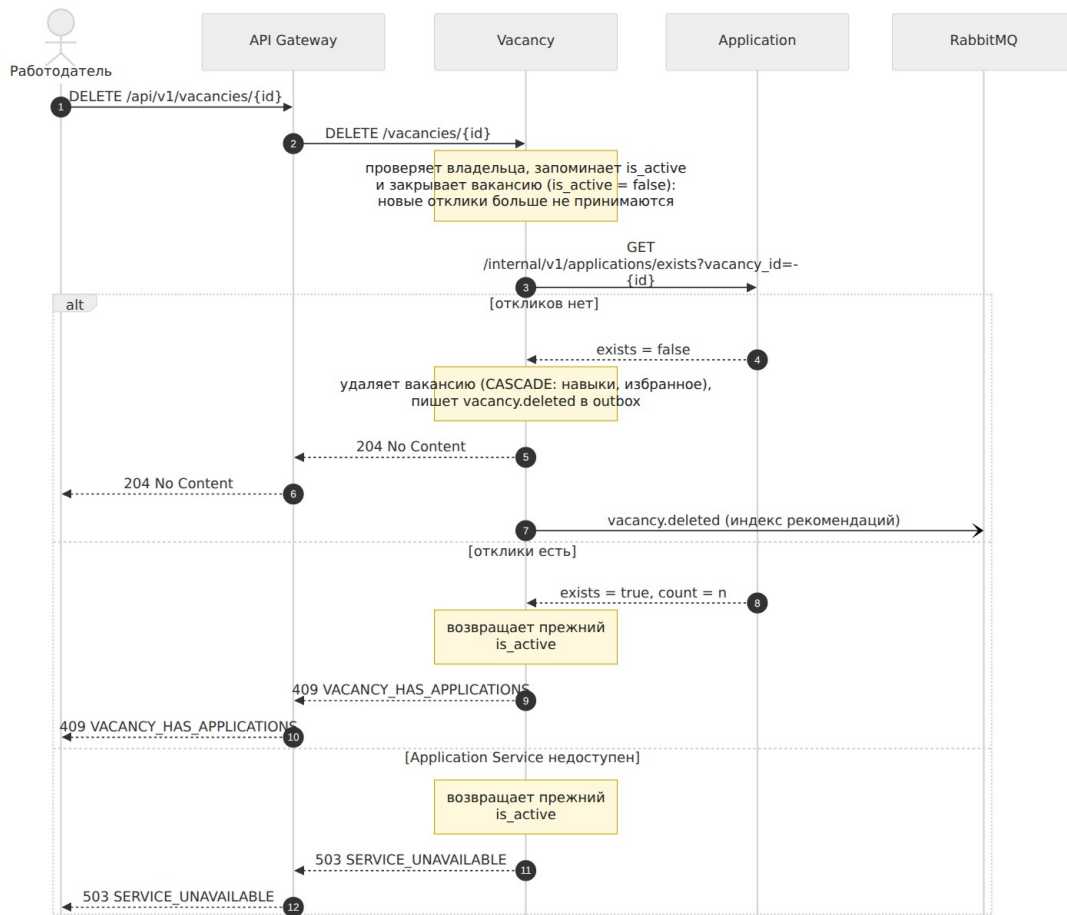


Рисунок 5 — Удаление вакансии

Рекомендованные вакансии для резюме (№48)

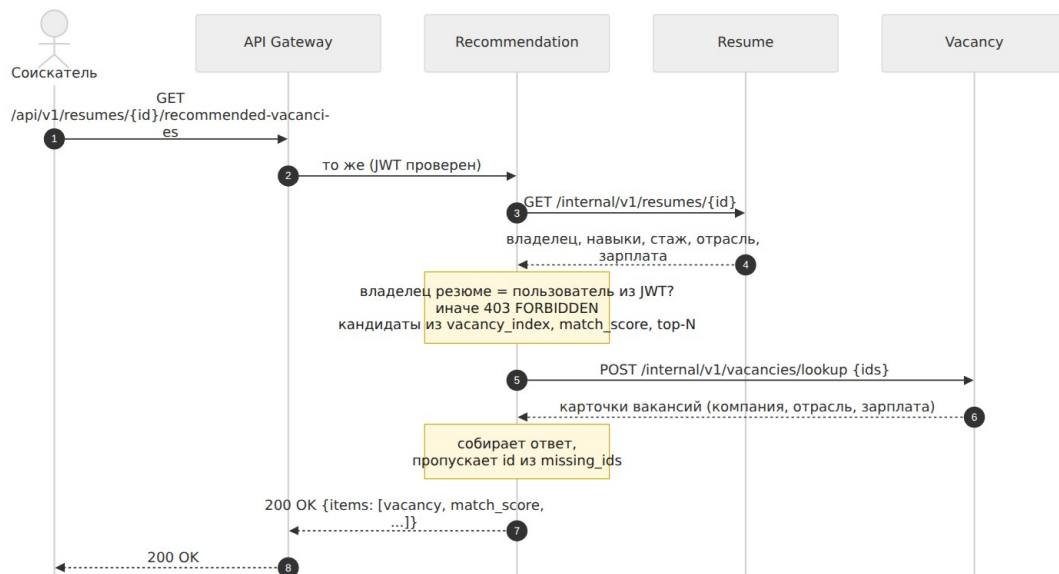
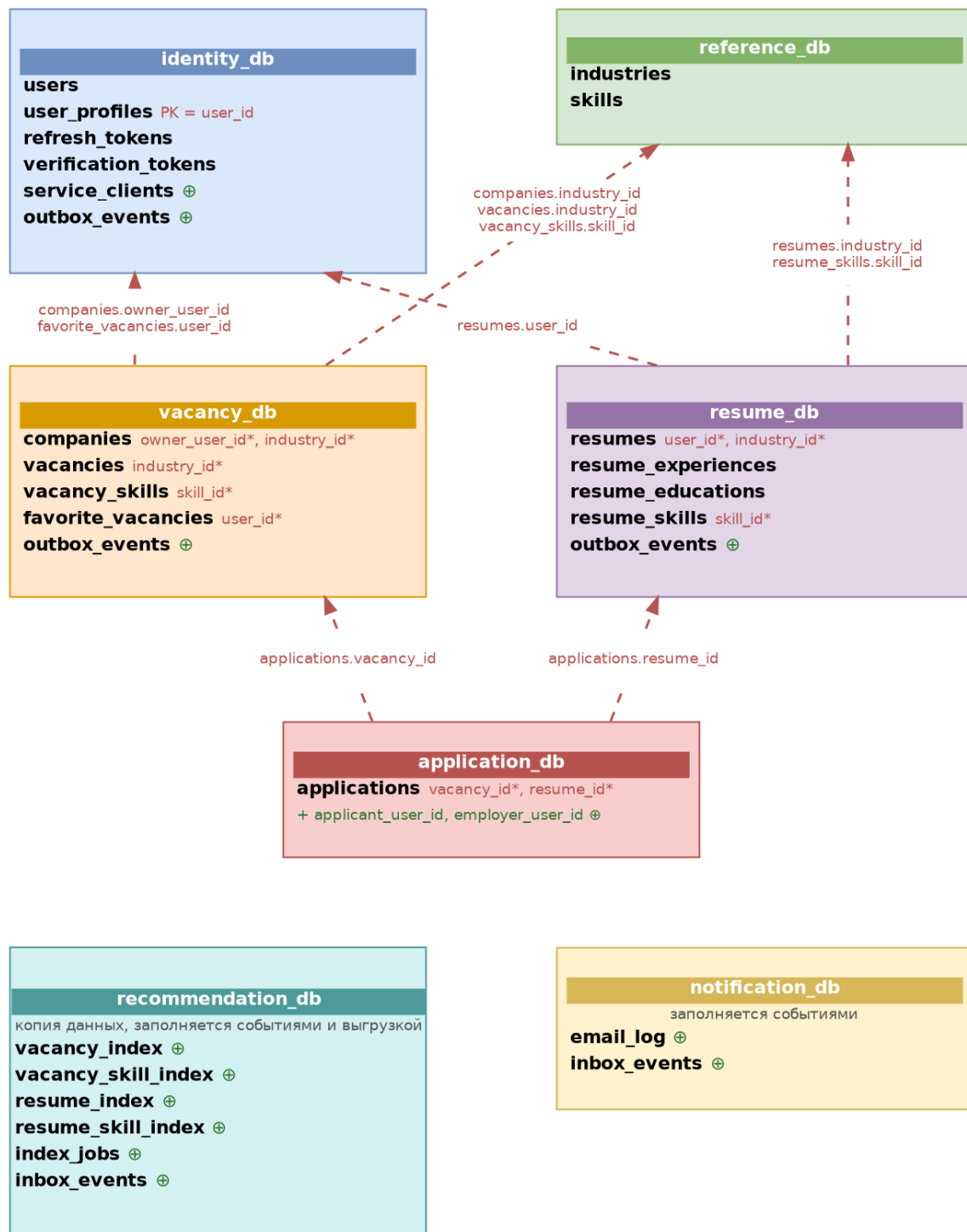


Рисунок 6 — Рекомендованные вакансии для резюме

4. Разделение базы данных

4.1. Принципы

- Каждая БД принадлежит одному сервису; в других сервисах нет ни подключений к ней, ни представлений, ни репликации её таблиц (кроме описанных копий данных в разделе 4.6).
- Внутри одной БД внешние ключи, каскады и ограничения уникальности сохраняются: 9 из 19 внешних ключей и все 8 ограничений UNIQUE остаются в своих БД без изменений.
- Ссылки между сервисами хранятся как обычные столбцы с идентификатором (логические ссылки): 10 внешних ключей превращаются в них.
- Схемы БД создаются миграциями, а не через `synchronize: true`, как в монолите, потому что автосинхронизация нескольких независимых схем небезопасна.



-- ► поле* хранит id из другой БД: логическая ссылка, внешнего ключа нет (сверка через API)

⊕ новая таблица или столбец (служебные и копии данных)

Связи внутри одной БД сохраняют внешние ключи и ON DELETE (CASCADE, RESTRICT)

Рисунок 7 — Разделение базы данных по сервисам

4.2. Состав баз данных

БД	Таблицы монолита	Новые таблицы	Логические ссылки
identity_db	users, user_profiles, refresh_tokens, verification_tokens	service_clients, outbox_events	нет
reference_db	industries, skills	нет	нет
vacancy_db	companies, vacancies, vacancy_skills, favorite_vacancies	outbox_events	companies.owner_user_id и favorite_vacancies.user_id → identity; companies.industry_id, vacancies.industry_id, vacancy_skills.skill_id → reference
resume_db	resumes, resume_experiences, resume_educations, resume_skills	outbox_events	resumes.user_id → identity; resumes.industry_id, resume_skills.skill_id → reference
application_db	applications (+ столбцы applicant_user_id, employer_user_id)	нет	vacancy_id → vacancy; resume_id → resume
recommendation_db	нет	vacancy_index, vacancy_skill_index, resume_index, resume_skill_index, index_jobs, inbox_events	нет (в индексе только копии идентификаторов)
notification_db	нет	email_log, inbox_events	нет

Итого в новой схеме 15 таблиц монолита и 12 новых (служебные и копии данных): outbox_events в трёх сервисах, inbox_events в двух, service_clients, index_jobs, четыре таблицы индекса и email_log.

4.3. Судьба внешних ключей монолита

№	Связь в монолите	Где теперь	Было	Как обеспечивается теперь
1	user_profiles.user_id → users	identity_db	CASCADE	Внешний ключ сохранён
2	verification_tokens.user_id → users	identity_db	CASCADE	Внешний ключ сохранён
3	refresh_tokens.user_id → users	identity_db	CASCADE	Внешний ключ сохранён
4	vacancies.company_id → companies	vacancy_db	RESTRICT	Внешний ключ сохранён (COMPANY_HAS_VACANCIES)
5	vacancy_skills.vacancy_id → vacancies	vacancy_db	CASCADE	Внешний ключ сохранён
6	favorite_vacancies.vacancy_id → vacancies	vacancy_db	CASCADE	Внешний ключ сохранён
7	resume_skills.resume_id → resumes	resume_db	CASCADE	Внешний ключ сохранён
8	resume_experiences.resume_id → resumes	resume_db	CASCADE	Внешний ключ сохранён
9	resume_educations.resume_id → resumes	resume_db	CASCADE	Внешний ключ сохранён
10	companies.owner_user_id → users	vacancy_db → identity_db	CASCADE	Логическая ссылка. Удаления пользователей в API нет; при его появлении Identity публикует user.deleted, а сервисы удаляют свои данные
11	favorite_vacancies.user_id → users	vacancy_db → identity_db	CASCADE	То же
12	resumes.user_id → users	resume_db → identity_db	CASCADE	То же
13	companies.industry_id → industries	vacancy_db → reference_db	RESTRICT	Проверка через industries/lookup при записи; отрасли не удаляются, а скрываются (is_published = false)
14	vacancies.industry_id → industries	vacancy_db → reference_db	RESTRICT	То же

№	Связь в монолите	Где теперь	Было	Как обеспечивается теперь
15	resumes.industry_id → industries	resume_db → reference_db	RESTRICT	То же
16	vacancy_skills.skill_id → skills	vacancy_db → reference_db	RESTRICT	Проверка через skills/lookup; навыки не удаляются
17	resume_skills.skill_id → skills	resume_db → reference_db	RESTRICT	То же
18	applications.vacancy_id → vacancies	application_db → vacancy_db	RESTRICT	Vacancy Service перед удалением спрашивает applications/exists (схема «закрыть, затем проверить»); сверка в Application
19	applications.resume_id → resumes	application_db → resume_db	RESTRICT	Resume Service перед удалением спрашивает applications/exists

4.4. Уникальность

Все восемь ограничений UNIQUE остаются внутри БД своего сервиса: users.email, refresh_tokens.token_hash (identity_db), industries.title, skills.name (reference_db), (vacancy_id, skill_id) и (user_id, vacancy_id) (vacancy_db), (resume_id, skill_id) (resume_db), (vacancy_id, resume_id) (application_db). Повторный отклик по-прежнему отсекается самой БД, а конфликт превращается в 409 APPLICATION_ALREADY_EXISTS.

4.5. Согласованность данных

Ситуация	Решение
Запись со ссылкой на чужие данные (отрасль, навык, вакансия, резюме)	Перед записью проверка существования через API (lookup, GET по id); ответ справочников кешируется на 10 минут
Удаление объекта, на который ссылаются другие БД	Проверка exists у владельца ссылок; для вакансии «закрыть, затем проверить», при ошибке возврат прежнего состояния
Гонка: отклик создан одновременно с удалением	Окно сокращено закрытием вакансии; остаток устраняет сверка целостности в Application Service (раз в час): отклики, чьи vacancy_id или resume_id вернулись в missing_ids, получают статус REJECTED
Изменение данных и публикация события	Outbox: событие записывается в одной транзакции с изменением, публикация повторяется до успеха
Повторная доставка события	Inbox: обработанные event_id запоминаются, повторное событие пропускается
Индекс рекомендаций отстаёт от источников	Допустимое отставание до 5 секунд (целевое); при расхождении полная перезагрузка POST /internal/v1/index/rebuild

4.6. Копии данных и денормализация

Ни одна копия не является источником истины; при расхождении верны данные владельца.

Что дублируется	Где	Зачем	Как обновляется
applicant_user_id, employer_user_id	application_db.applications	Права на отклик проверяются локально, без вызовов; фильтр «мои отклики»	Заполняется при создании отклика из ответов Resume и Vacancy; не меняется
Поля вакансий и резюме для подсчёта соответствия	recommendation_db (таблицы *_index)	Подбор без обращений к сервисам на каждый запрос	События vacancy.*, resume.*; полная загрузка через export
Названия отраслей и навыков	Кеш в памяти Vacancy и Resume	Карточки без вызова Reference на каждый запрос	TTL 10 минут, при ошибке используется устаревшее значение

4.7. Новые служебные таблицы

Таблица	Сервис	Назначение	Основные поля
outbox_events	identity, vacancy, resume	Надёжная публикация событий	id, type, payload, created_at, published_at
inbox_events	recommendation, notification	Идемпотентность потребителя	event_id, processed_at
service_clients	identity	Клиенты для сервисных токенов	client_id, secret_hash, allowed_audiences, is_active
vacancy_index, vacancy_skill_index	recommendation	Копия вакансий для подбора	vacancy_id, industry_id, salary_from, salary_to, min_experience_years, is_active, updated_at; (vacancy_id, skill_id, is_required)
resume_index, resume_skill_index	recommendation	Копия опубликованных резюме для подбора	resume_id, user_id, industry_id, salary_exp_from, salary_exp_to, experience_periods, updated_at; (resume_id, skill_id)
index_jobs	recommendation	Задачи перестроения индекса	id, scope, status, processed_items, started_at, finished_at
email_log	notification	Журнал отправки (без текста кода)	id, to_email, template, status, attempts, created_at, sent_at

5. Межсервисное API

5.1. Общие соглашения

Внутреннее API описано в файле `openapi-internal.yaml` (OpenAPI 3.0.3): 17 операций, 44 схемы данных. Оно отделено от публичного API и снаружи недоступно.

- Адреса. Все эндпоинты начинаются с `/internal/v1`. Каждый сервис слушает свой хост и порт (`identity-service:8001` и далее), адреса указаны в `servers` у каждого пути.
- Аутентификация. Заголовок `Authorization: Bearer <сервисный токен>`. Без токена доступны только выдача токена, JWKS и `health`.
- Заголовок `X-Request-Id` передаётся по всей цепочке вызовов и возвращается в поле `request_id` ошибок; если его нет, сервис создаёт значение сам.
- Пакетные запросы (`lookup`) принимают до 100 идентификаторов; отсутствующие возвращаются в `missing_ids`, ошибкой это не считается.
- Выгрузки (`export`) используют курсорную пагинацию: параметры `updated_after`, `cursor`, `limit` (до 500) и поле `next_cursor` в ответе.
- Формат ошибок тот же, что в публичном API: `code`, `message`, `details`, `request_id`.
- Версия задаётся в пути (`/v1`); несовместимые изменения выпускаются как `/v2`, старая версия поддерживается на время перехода.

5.2. Список эндпоинтов

№	Сервис	Метод и путь	Назначение	Кто вызывает
1	Identity	POST <code>/internal/v1/auth/service-token</code>	Выдача сервисного токена	identity, reference, vacancy, resume, application, recommendation, notification, api-gateway, ops-tools

№	Сервис	Метод и путь	Назначение	Кто вызывает
2	Identity	GET /internal/v1/.well-known/jwks.json	Публичные ключи для проверки JWT	api-gateway, identity, reference, vacancy, resume, application, recommendation, notification
3	Identity	POST /internal/v1/users/lookup	Пакетное получение пользователей	application, recommendation
4	Reference	POST /internal/v1/industries/lookup	Пакетное получение отраслей	vacancy, resume
5	Reference	POST /internal/v1/skills/lookup	Пакетное получение навыков	vacancy, resume
6	Vacancy	GET /internal/v1/vacancies/{vacancyId}	Вакансия для внутреннего использования	application, recommendation
7	Vacancy	POST /internal/v1/vacancies/lookup	Пакетное получение вакансий	application, recommendation
8	Vacancy	GET /internal/v1/vacancies/export	Выгрузка вакансий для индекса рекомендаций	recommendation
9	Resume	GET /internal/v1/resumes/{resumeId}	Резюме для внутреннего использования	application, recommendation
10	Resume	POST /internal/v1/resumes/lookup	Пакетное получение резюме	application, recommendation
11	Resume	GET /internal/v1/resumes/export	Выгрузка резюме для индекса рекомендаций	recommendation
12	Application	GET /internal/v1/applications/exists	Есть ли отклики на вакансию или резюме	vacancy, resume
13	Application	GET /internal/v1/applications/access-check	Доступ работодателя к резюме через отклик	resume
14	Recommendation	POST /internal/v1/index/rebuild	Перестроение индекса рекомендаций	ops-tools
15	Recommendation	GET /internal/v1/index/rebuild/{jobId}	Состояние задачи перестроения	ops-tools
16	Notification	GET /internal/v1/dev/mailbox	Тестовый почтовый ящик	api-gateway
17	Common	GET /internal/v1/health	Состояние сервиса	api-gateway

Ниже для каждого эндпоинта приведены назначение, параметры, поля ответа, возможные ошибки и пример. Полное описание со всеми схемами и примерами ошибок находится в `openapi-internal.yaml`.

5.3. Описание эндпоинтов

5.3.1. POST /internal/v1/auth/service-token — Выдача сервисного токена

Аутентификация сервиса по схеме `client credentials`. Сервис получает короткоживущий JWT (5 минут), который подставляет в заголовок `Authorization` при вызове других сервисов. Токен привязан к получателю (`aud`), поэтому токен для `vacancy-service` не подойдет для `resume-service`. Вызывающий сервис кеширует токен до истечения срока.

Кто вызывает: `identity-service`, `reference-service`, `vacancy-service`, `resume-service`, `application-service`, `recommendation-service`, `notification-service`, `api-gateway`, `ops-tools`. Доступ: публичный внутри сети (токен не нужен). Успешный ответ: 200 OK.

Где используется: Все вызовы между сервисами (получение токена перед первым обращением и после истечения).

Тело запроса (ServiceTokenRequest):

Поле	Тип	Обяз.	Описание
client_id	enum (ClientName)	да	Имя вызывающего сервиса
client_secret	string	да	Секрет клиента (из конфигурации сервиса, не из кода) (длина 16–128)
audience	enum (AudienceName)	да	Сервис, для которого нужен токен

Тело ответа (ServiceToken):

Поле	Тип	Обяз.	Описание
access_token	string	да	JWT: sub = имя клиента, aud = имя сервиса-получателя
token_type	string	да	Тип токена
expires_in	integer	да	Срок жизни, секунды (5 минут)
audience	enum (AudienceName)	да	Сервис, для которого выдан токен

Ошибки:

HTTP	Код	Когда возникает
401	INVALID_CLIENT	client_id или client_secret не совпадают с зарегистрированными
403	AUDIENCE_NOT_ALLOWED	Для пары client_id и audience токен не выдаётся
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
429	TOO_MANY_REQUESTS	Превышен лимит запросов; повторите позже (заголовок Retry-After)
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример запроса:

```
{
  "client_id": "application-service",
  "client_secret": "s3cr3t-of-application-service",
  "audience": "vacancy-service"
}
```

Пример ответа:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6ImxIn0...",
  "token_type": "Bearer",
  "expires_in": 300,
  "audience": "vacancy-service"
}
```

5.3.2. GET /internal/v1/.well-known/jwks.json — Публичные ключи для проверки JWT

Отдаёт публичные ключи, которыми подписаны access-токены пользователей и сервисные токены. Шлюз и сервисы кешируют набор и обновляют его при появлении неизвестного kid. Благодаря этому сервисы проверяют подпись самостоятельно и не обращаются в Identity на каждый запрос.

Кто вызывает: api-gateway, identity-service, reference-service, vacancy-service, resume-service, application-service, recommendation-service, notification-service. Доступ: публичный внутри сети (токен не нужен). Успешный ответ: 200 OK.

Где используется: Проверка JWT в шлюзе и во всех сервисах (все защищённые операции публичного API).

Тело ответа (Jwks):

Поле	Тип	Обяз.	Описание
keys	array<JwkKey>	да	Действующие ключи; при ротации в набор входят старый и новый

Ошибки:

HTTP	Код	Когда возникает
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "keys": [
    {
      "kty": "RSA",
      "use": "sig",
      "alg": "RS256",
      "kid": "2026-09-key-1",
      "n": "sXchDaQebHnPiGvyDOAT4saGEUetSyo9MKLOoWFsueri23bOdgWp4Dy1Wl...",
      "e": "AQAB"
    }
  ]
}
```

5.3.3. POST /internal/v1/users/lookup — Пакетное получение пользователей

Возвращает имена (и при include_contacts = true контакты) пользователей по списку id. Нужен, чтобы собрать карточку кандидата в откликах и рекомендациях, потому что профили хранятся только в Identity. Контакты получает лишь application-service: работодатель видит их в карточке отклика, но не в рекомендациях.

Кто вызывает: application-service, recommendation-service. Доступ: сервисный токен, aud = identity-service. Успешный ответ: 200 OK.

Где используется: №39 отклики на вакансию, №41 мои отклики, №42 просмотр отклика, №49 рекомендованные кандидаты.

Тело запроса (UserLookupRequest):

Поле	Тип	Обяз.	Описание
ids	array<string (uuid)>	да	Идентификаторы пользователей (от 1 до 100) (1–100 элементов)
include_contacts	boolean	нет	Вернуть контакты (email, телефон, город). Разрешено только application-service (по умолчанию: false)

Тело ответа (UserLookupResponse):

Поле	Тип	Обяз.	Описание
items	array<InternalUser>	да	Найденные пользователи
missing_ids	array<string (uuid)>	да	Идентификаторы, которых нет в Identity

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан не для identity-service либо include_contacts = true запрошен не application-service
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример запроса:

```
{
  "ids": [
    "531c4d1e-d877-50be-bf27-768458436c9e",
    "7cfe1d2c-9559-516e-a053-28eb980eff75"
  ],
  "include_contacts": false
}
```

Пример ответа:

```
{
  "items": [
    {
      "id": "531c4d1e-d877-50be-bf27-768458436c9e",
      "role": "JOBSEEKER",
      "is_active": true,
      "last_name": "Дементьев",
      "first_name": "Антон",
      "middle_name": "Павлович",
      "email": "anton@example.com",
      "phone": "+7 900 000-00-00",
      "city": "Санкт-Петербург"
    }
  ],
  "missing_ids": []
}
```

5.3.4. POST /internal/v1/industries/lookup — Пакетное получение отраслей

Проверяет, что отрасли существуют и опубликованы, и возвращает их названия. Вызывается при создании и изменении компаний, вакансий и резюме, а также при сборке карточек. Ответ можно кешировать на 10 минут: справочник меняется редко.

Кто вызывает: vacancy-service, resume-service. Доступ: сервисный токен, aud = reference-service. Успешный ответ: 200 OK.

Где используется: №16, №18 резюме; №28, №30 компании; №34, №36 вакансии; названия отраслей во всех карточках.

Тело запроса (IdsRequest):

Поле	Тип	Обяз.	Описание
ids	array<string (uuid)>	да	Идентификаторы (от 1 до 100). Повторы игнорируются (1–100 элементов)

Тело ответа (IndustryLookupResponse):

Поле	Тип	Обяз.	Описание
items	array<ReferenceIndustry>	да	Найденные отрасли
missing_ids	array<string (uuid)>	да	Идентификаторы, которых нет в справочнике

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример запроса:

```
{
  "ids": [
    "4fc79438-2514-51a8-9796-426af8d8f73d",
    "43df9396-dbd5-598c-ba46-a84fb4e24f64"
  ]
}
```

Пример ответа:

```
{
  "items": [
    {
      "id": "ef39a9bf-1aa7-521e-870a-46190bc1241a",
      "title": "Информационные технологии",
      "is_published": true
    }
  ],
  "missing_ids": []
}
```

5.3.5. POST /internal/v1/skills/lookup — Пакетное получение навыков

Проверяет, что навыки существуют в справочнике, и возвращает их названия. Вызывается при задании навыков резюме и вакансии; ответ кешируется на 10 минут.

Кто вызывает: vacancy-service, resume-service. Доступ: сервисный токен, aud = reference-service. Успешный ответ: 200 OK.

Где используется: №26 навыки резюме, №38 навыки вакансии; названия навыков в карточках.

Тело запроса (IdsRequest):

Поле	Тип	Обяз.	Описание
ids	array<string (uuid)>	да	Идентификаторы (от 1 до 100). Повторы игнорируются (1–100 элементов)

Тело ответа (SkillLookupResponse):

Поле	Тип	Обяз.	Описание
items	array<Skill>	да	Найденные навыки
missing_ids	array<string (uuid)>	да	Идентификаторы, которых нет в справочнике

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример запроса:

```
{
  "ids": [
    "4fc79438-2514-51a8-9796-426af8d8f73d",
    "43df9396-dbd5-598c-ba46-a84fb4e24f64"
  ]
}
```

Пример ответа:

```
{
  "items": [
    {
      "id": "e2ee04fb-d7c7-5d93-8439-208548ade0fa",
      "name": "Python"
    }
  ],
  "missing_ids": []
}
```

5.3.6. GET /internal/v1/vacancies/{vacancyId} — Вакансия для внутреннего использования

Возвращает вакансию вместе с компанией, владельцем компании, отраслью и навыками. В отличие от публичного API возвращает и закрытые вакансии, чтобы вызывающий сервис сам решил, что делать (например, отклонить отклик с VACANCY_NOT_ACTIVE). Свежие данные без кеша.

Кто вызывает: application-service, recommendation-service. Доступ: сервисный токен, aud = vacancy-service. Успешный ответ: 200 OK.

Где используется: №39 (проверка владельца вакансии), №40 отклик на вакансию, №49 рекомендованные кандидаты.

Параметр	Где	Тип	Обяз.	Описание
vacancyId	path	string (uuid)	да	Идентификатор вакансии

Тело ответа (InternalVacancy):

Поле	Тип	Обяз.	Описание
id	string (uuid)	да	Идентификатор вакансии
title	string	да	Название вакансии
company	object (InternalCompany)	да	Компания
industry	object (IndustryRef)	да	Отрасль
salary_from	integer, null	да	Зарплата от
salary_to	integer, null	да	Зарплата до
min_experience_years	integer	да	Минимальный опыт, лет
is_active	boolean	да	Вакансия открыта для откликов
skills	array<VacancySkillRef>	да	Навыки вакансии
created_at	string (date-time)	да	Дата создания
updated_at	string (date-time)	да	Дата последнего изменения

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
404	VACANCY_NOT_FOUND	Вакансия с указанным id не существует
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
504	UPSTREAM_TIMEOUT	Вызов зависимого сервиса не уложился в тайм-аут
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "id": "4fc79438-2514-51a8-9796-426af8d8f73d",
  "title": "Python-разработчик (Backend)",
  "company": {
    "id": "677bf595-770f-5b5e-acff-cc36ffb6020c",
    "name": "Технологии Будущего",
    "website_url": "https://future.example.com",
    "owner_user_id": "7cfe1d2c-9559-516e-a053-28eb980eff75"
  },
  "industry": {
    "id": "ef39a9bf-1aa7-521e-870a-46190bc1241a",
    "title": "Информационные технологии"
  },
  "salary_from": 150000,
  "salary_to": 250000,
  "min_experience_years": 1,
  "is_active": true,
  "skills": [
    {
      "skill": {
        "id": "e2ee04fb-d7c7-5d93-8439-208548ade0fa",
        "name": "Python"
      },
      "is_required": true
    }
  ],
  "created_at": "2026-09-18T10:15:30Z",
  "updated_at": "2026-09-19T08:00:00Z"
}
```

5.3.7. POST /internal/v1/vacancies/lookup — Пакетное получение вакансий

Возвращает карточки вакансий по списку id одним запросом (без N+1). Отсутствующие id перечисляются в missing_ids, ошибкой это не считается: так вызывающий сервис видит отклики и рекомендации, указывающие на удалённые вакансии.

Кто вызывает: application-service, recommendation-service. Доступ: сервисный токен, aud = vacancy-service. Успешный ответ: 200 OK.

Где используется: №39, №41, №42 карточки откликов; №48 рекомендованные вакансии; сверка целостности откликов.

Тело запроса (IdsRequest):

Поле	Тип	Обяз.	Описание
ids	array<string (uuid)>	да	Идентификаторы (от 1 до 100). Повторы игнорируются (1–100 элементов)

Тело ответа (VacancyLookupResponse):

Поле	Тип	Обяз.	Описание
items	array<InternalVacancy>	да	Найденные вакансии
missing_ids	array<string (uuid)>	да	Идентификаторы, которых нет в Vacancy Service

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
504	UPSTREAM_TIMEOUT	Вызов зависимого сервиса не уложился в тайм-аут
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример запроса:

```
{
  "ids": [
    "4fc79438-2514-51a8-9796-426af8d8f73d",
    "43df9396-dbd5-598c-ba46-a84fb4e24f64"
  ]
}
```

Пример ответа:

```
{
  "items": [
    {
      "id": "4fc79438-2514-51a8-9796-426af8d8f73d",
      "title": "Python-разработчик (Backend)",
      "company": {
        "id": "677bf595-770f-5b5e-acff-cc36ffb6020c",
        "name": "Технологии Будущего",
        "website_url": "https://future.example.com",
        "owner_user_id": "7cfe1d2c-9559-516e-a053-28eb980eff75"
      },
      "industry": {
        "id": "ef39a9bf-1aa7-521e-870a-46190bc1241a",
        "title": "Информационные технологии"
      },
      "salary_from": 150000,
      "salary_to": 250000,
      "min_experience_years": 1,
      "is_active": true,
      "skills": [
        {
          "skill": {
            "id": "e2ee04fb-d7c7-5d93-8439-208548ade0fa",
            "name": "Python"
          },
          "is_required": true
        }
      ],
      "created_at": "2026-09-18T10:15:30Z",
      "updated_at": "2026-09-19T08:00:00Z"
    }
  ],
  "missing_ids": []
}
```

5.3.8. GET /internal/v1/vacancies/export — Выгрузка вакансий для индекса рекомендаций

Постраничная выгрузка данных, нужных для подсчёта соответствия. Используется для первичной загрузки индекса при запуске Recommendation Service и для перестроения индекса. Дальше индекс поддерживается событиями vacancy.upserted и vacancy.deleted.

Кто вызывает: recommendation-service. Доступ: сервисный токен, aud = vacancy-service. Успешный ответ: 200 ОК.

Где используется: Первичная загрузка и перестроение индекса рекомендаций (№48, №49).

Параметр	Где	Тип	Обяз.	Описание
updated_after	query	string (date-time)	нет	Вернуть записи, изменённые после этого момента (для инкрементальной загрузки)
cursor	query	string	нет	Курсор из next_cursor предыдущей страницы
limit	query	integer	нет	Размер страницы выгрузки (максимум 500) (значение 1–500; по умолчанию: 200)

Тело ответа (VacancyExportPage):

Поле	Тип	Обяз.	Описание
items	array<VacancyIndexRecord>	да	Записи страницы
next_cursor	string, null	да	Курсор следующей страницы; null — данных больше нет

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "items": [
    {
      "vacancy_id": "4fc79438-2514-51a8-9796-426af8d8f73d",
      "industry_id": "ef39a9bf-1aa7-521e-870a-46190bc1241a",
      "salary_from": 150000,
      "salary_to": 250000,
      "min_experience_years": 1,
      "is_active": true,
      "skills": [
        {
          "skill_id": "e2ee04fb-d7c7-5d93-8439-208548ade0fa",
          "is_required": true
        }
      ],
      "created_at": "2026-09-18T10:15:30Z",
      "updated_at": "2026-09-19T08:00:00Z"
    }
  ],
  "next_cursor": "eyJ1Ijo1MjAyNi0wOS0xOVQwODowMDowMFoifQ"
}
```

5.3.9. GET /internal/v1/resumes/{resumeId} — Резюме для внутреннего использования

Возвращает резюме с владельцем, отраслью, навыками и стажем. При view = detail добавляются места работы и образование. Возвращает и неопубликованные резюме: решение об отказе (RESUME_NOT_PUBLISHED, FORBIDDEN) принимает вызывающий сервис. Свежие данные без кеша.

Кто вызывает: application-service, recommendation-service. Доступ: сервисный токен, aud = resume-service. Успешный ответ: 200 OK.

Где используется: №40 отклик на вакансию (владелец и публикация резюме), №48 рекомендованные вакансии.

Параметр	Где	Тип	Обяз.	Описание
resumeId	path	string (uuid)	да	Идентификатор резюме
view	query	enum: short detail	нет	Уровень детализации: short или detail (по умолчанию: short)

Тело ответа (InternalResume):

Поле	Тип	Обяз.	Описание
id	string (uuid)	да	Идентификатор резюме
user_id	string (uuid)	да	Владелец резюме
title	string	да	Название резюме
summary	string, null	да	О себе
industry	object (IndustryRef)	да	Отрасль
salary_exp_from	integer, null	да	Ожидаемая зарплата от
salary_exp_to	integer, null	да	Ожидаемая зарплата до
is_published	boolean	да	Резюме опубликовано
total_experience_months	integer	да	Суммарный стаж, месяцев (считается по местам работы)
skills	array<Skill>	да	Навыки резюме
experiences	array<ResumeExperience>	нет	Места работы. Только при view = detail
educations	array<ResumeEducation>	нет	Образование. Только при view = detail
created_at	string (date-time)	да	Дата создания
updated_at	string (date-time)	да	Дата изменения

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
404	RESUME_NOT_FOUND	Резюме с указанным id не существует
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
504	UPSTREAM_TIMEOUT	Вызов зависимого сервиса не уложился в тайм-аут
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

5.3.10. POST /internal/v1/resumes/lookup — Пакетное получение резюме

Возвращает карточки резюме по списку id одним запросом: краткие (view = short) для списков и полные (view = detail) для карточки отклика.

Кто вызывает: application-service, recommendation-service. Доступ: сервисный токен, aud = resume-service. Успешный ответ: 200 OK.

Где используется: №39, №41 списки откликов (short); №42 карточка отклика (detail); №49 рекомендованные кандидаты (short).

Тело запроса (ResumeLookupRequest):

Поле	Тип	Обяз.	Описание
ids	array<string (uuid)>	да	Идентификаторы резюме (от 1 до 100) (1–100 элементов)
view	enum (ResumeView)	нет	Уровень детализации (по умолчанию: short)

Тело ответа (ResumeLookupResponse):

Поле	Тип	Обяз.	Описание
items	array<InternalResume>	да	Найденные резюме
missing_ids	array<string (uuid)>	да	Идентификаторы, которых нет в Resume Service

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
504	UPSTREAM_TIMEOUT	Вызов зависимого сервиса не уложился в тайм-аут
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример запроса:

```
{
  "ids": [
    "06b39995-c3d8-5fcb-831f-7d1fd3421f6e"
  ],
  "view": "short"
}
```

5.3.11. GET /internal/v1/resumes/export — Выгрузка резюме для индекса рекомендаций

Постраничная выгрузка опубликованных резюме (id, владелец, отрасль, зарплата, периоды работы, навыки). Назначение то же, что у выгрузки вакансий: первичная загрузка и перестроение индекса.

Кто вызывает: recommendation-service. Доступ: сервисный токен, aud = resume-service. Успешный ответ: 200 OK.

Где используется: Первичная загрузка и перестроение индекса рекомендаций (№48, №49).

Параметр	Где	Тип	Обяз.	Описание
updated_after	query	string (date-time)	нет	Вернуть записи, изменённые после этого момента (для инкрементальной загрузки)
cursor	query	string	нет	Курсор из next_cursor предыдущей страницы
limit	query	integer	нет	Размер страницы выгрузки (максимум 500) (значение 1–500; по умолчанию: 200)

Тело ответа (ResumeExportPage):

Поле	Тип	Обяз.	Описание
items	array<ResumeIndexRecord>	да	Записи страницы
next_cursor	string, null	да	Курсор следующей страницы; null — данных больше нет

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "items": [
    {
      "resume_id": "06b39995-c3d8-5fcb-831f-7d1fd3421f6e",
      "user_id": "531c4d1e-d877-50be-bf27-768458436c9e",
      "industry_id": "ef39a9bf-1aa7-521e-870a-46190bc1241a",
      "salary_exp_from": 150000,
      "salary_exp_to": 220000,
      "is_published": true,
      "experience_periods": [
        {
          "start_date": "2024-09-01",
          "end_date": null
        }
      ],
      "skill_ids": [
        "e2ee04fb-d7c7-5d93-8439-208548ade0fa",
        "982780ef-4d5a-5e5e-8e6b-8a3eb4d480be"
      ],
      "updated_at": "2026-09-19T08:00:00Z"
    }
  ],
  "next_cursor": null
}
```

5.3.12. GET /internal/v1/applications/exists — Есть ли отклики на вакансию или резюме

Заменяет правило ON DELETE RESTRICT, которое в монолите обеспечивала БД: Vacancy Service и Resume Service спрашивают, есть ли отклики, прежде чем удалить вакансию или резюме. Для вакансии проверка выполняется по схеме «сначала закрыть, потом проверить»: вакансия предварительно переводится в `is_active = false`, чтобы новые отклики не появлялись.

Кто вызывает: vacancy-service, resume-service. Доступ: сервисный токен, aud = application-service. Успешный ответ: 200 OK.

Где используется: №37 удаление вакансии, №19 удаление резюме.

Параметр	Где	Тип	Обяз.	Описание
vacancy_id	query	string (uuid)	нет	Идентификатор вакансии. Нужно передать ровно один из параметров vacancy_id и resume_id
resume_id	query	string (uuid)	нет	Идентификатор резюме. Нужно передать ровно один из параметров vacancy_id и resume_id

Тело ответа (ExistsResponse):

Поле	Тип	Обяз.	Описание
exists	boolean	да	Есть ли хотя бы один отклик
count	integer	да	Сколько откликов найдено

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Не передан ни один из параметров vacancy_id, resume_id либо переданы оба
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "exists": true,
  "count": 3
}
```

5.3.13. GET /internal/v1/applications/access-check — Доступ работодателя к резюме через отклик

Отвечает, есть ли отклик с этим резюме на вакансию компании этого работодателя. Resume Service использует ответ, чтобы решить, показывать ли работодателю резюме (в монолите это был JOIN applications, vacancies, companies). Для этого Application Service хранит у отклика владельца вакансии (employer_user_id).

Кто вызывает: resume-service. Доступ: сервисный токен, aud = application-service. Успешный ответ: 200 OK.

Где используется: №17 просмотр резюме работодателем.

Параметр	Где	Тип	Обяз.	Описание
resume_id	query	string (uuid)	да	Идентификатор резюме
employer_user_id	query	string (uuid)	да	Пользователь-работодатель, который открывает резюме

Тело ответа (AccessCheckResponse):

Поле	Тип	Обяз.	Описание
allowed	boolean	да	Есть отклик этого резюме на вакансию компании работодателя

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "allowed": true
}
```

5.3.14. POST /internal/v1/index/rebuild — Перестроение индекса рекомендаций

Запускает фоновую задачу, которая заново загружает индекс из выгрузок Vacancy Service и Resume Service. Нужна при первом запуске сервиса, после потери данных или расхождения с источниками. Одновременно выполняется одна задача на область.

Кто вызывает: ops-tools. Доступ: сервисный токен, aud = recommendation-service. Успешный ответ: 202 Accepted.

Где используется: Миграция: начальная загрузка индекса; эксплуатация.

Тело запроса (IndexRebuildRequest):

Поле	Тип	Обяз.	Описание
scope	enum (IndexScope)	нет	Что перестраивать (по умолчанию: ALL)

Тело ответа (IndexRebuildJob):

Поле	Тип	Обяз.	Описание
job_id	string (uuid)	да	Идентификатор задачи
scope	enum (IndexScope)	да	Что перестраивается
status	enum (JobStatus)	да	Состояние задачи
processed_items	integer	да	Сколько записей уже загружено
started_at	string (date-time)	да	Время запуска
finished_at	string (date-time), null	да	Время завершения

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
409	INDEX_REBUILD_IN_PROGRESS	Для этой области уже запущена задача в состоянии QUEUED или RUNNING
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример запроса:

```
{
  "scope": "ALL"
}
```

Пример ответа:

```
{
  "job_id": "c8eb8aee-b7ea-5270-8810-4f17e9c0c928",
  "scope": "ALL",
  "status": "RUNNING",
  "processed_items": 1200,
  "started_at": "2026-09-18T10:15:30Z",
  "finished_at": null
}
```

5.3.15. GET /internal/v1/index/rebuild/{jobId} — Состояние задачи перестроения

Возвращает состояние задачи перестроения индекса: сколько записей загружено и закончена ли загрузка.

Кто вызывает: ops-tools. Доступ: сервисный токен, aud = recommendation-service. Успешный ответ: 200 OK.

Где используется: Миграция: контроль начальной загрузки индекса.

Параметр	Где	Тип	Обяз.	Описание
jobId	path	string (uuid)	да	Идентификатор задачи

Тело ответа (IndexRebuildJob):

Поле	Тип	Обяз.	Описание
job_id	string (uuid)	да	Идентификатор задачи
scope	enum (IndexScope)	да	Что перестраивается
status	enum (JobStatus)	да	Состояние задачи
processed_items	integer	да	Сколько записей уже загружено
started_at	string (date-time)	да	Время запуска
finished_at	string (date-time), null	да	Время завершения

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
404	JOB_NOT_FOUND	Задача перестроения индекса с указанным id не существует
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "job_id": "c8eb8aee-b7ea-5270-8810-4f17e9c0c928",
  "scope": "ALL",
  "status": "RUNNING",
  "processed_items": 1200,
  "started_at": "2026-09-18T10:15:30Z",
  "finished_at": null
}
```

5.3.16. GET /internal/v1/dev/mailbox — Тестовый почтовый ящик

ТОЛЬКО ДЛЯ ТЕСТОВЫХ СТЕНДОВ. Возвращает письма, «отправленные» на адрес, вместе с 6-значным кодом. Работает при MAIL_DEBUG=true; в продуктивной среде отвечает 404 NOT_FOUND. Шлюз пробрасывает его как /api/v1/dev/mailbox только на тестовом стенде: это позволяет сценарию Postman из ДЗ 3 проходить подтверждение email автоматически.

Кто вызывает: api-gateway. Доступ: сервисный токен, aud = notification-service. Успешный ответ: 200 OK.

Где используется: Автотесты (сценарий Postman из ДЗ 3) на тестовом стенде.

Параметр	Где	Тип	Обяз.	Описание
email	query	string (email)	да	Адрес, письма которому нужно получить

Тело ответа (MailboxResponse):

Поле	Тип	Обяз.	Описание
items	array<MailboxItem>	да	Письма

Ошибки:

HTTP	Код	Когда возникает
401	SERVICE_UNAUTHORIZED	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
403	SERVICE_FORBIDDEN	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
404	NOT_FOUND	Эндпоинт отсутствует или отключён (например, dev/mailbox при MAIL_DEBUG=false)
422	VALIDATION_ERROR	Параметры или тело запроса не прошли проверку; подробности в details
503	SERVICE_UNAVAILABLE	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "items": [
    {
      "to": "anton@example.com",
      "subject": "Подтверждение email",
      "template": "EMAIL_VERIFICATION",
      "code": "482913",
      "sent_at": "2026-09-18T10:15:30Z"
    }
  ]
}
```

5.3.17. GET /internal/v1/health — Состояние сервиса

Есть у каждого из семи сервисов (адреса в servers). Возвращает готовность сервиса и его зависимостей. Шлюз опрашивает все сервисы и собирает из ответов публичный GET /api/v1/health; этот же эндпоинт используют проверки готовности (readiness) оркестратора.

Кто вызывает: api-gateway. Доступ: публичный внутри сети (токен не нужен). Успешный ответ: 200 OK.

Где используется: №50 проверка состояния сервиса.

Тело ответа (ServiceHealth):

Поле	Тип	Обяз.	Описание
service	string	да	Имя сервиса
status	enum (HealthStatus)	да	Общее состояние
version	string	да	Версия сервиса
database	enum: ok down	да	Доступность собственной БД
broker	enum: ok down, null	да	Доступность брокера сообщений; null, если сервис его не использует

Ошибки:

HTTP	Код	Когда возникает
503	SERVICE_UNAVAILABLE	Собственная БД сервиса недоступна
500	INTERNAL_ERROR	Непредвиденная ошибка на стороне сервиса

Пример ответа:

```
{
  "service": "vacancy-service",
  "status": "ok",
  "version": "1.0.0",
  "database": "ok",
  "broker": "ok"
}
```

5.4. Перечисления

Перечисление	Значения	Где используется
Role	JOBSEEKER, EMPLOYER	роль пользователя в InternalUser
ClientName	api-gateway, identity-service, reference-service, vacancy-service, resume-service, application-service, recommendation-service, notification-service, ops-tools	client_id при выдаче сервисного токена
AudienceName	семь сервисов (все значения ClientName, кроме api-gateway и ops-tools)	audience сервисного токена
ResumeView	short, detail	уровень детализации резюме
IndexScope	VACANCIES, RESUMES, ALL	перестроение индекса
JobStatus	QUEUED, RUNNING, DONE, FAILED	задача перестроения индекса
HealthStatus	ok, degraded	состояние сервиса

5.5. Контракты событий

Все события передаются как JSON в конверте с полями event_id (уникальный идентификатор, по нему работает inbox), type, version (версия формата), occurred_at и payload. Ключ маршрутизации в RabbitMQ совпадает с type.

5.5.1. identity.email_requested

```
{
  "event_id": "90e0c3a9-8135-5e5f-bd6f-dcc63ecee36b",
  "type": "identity.email_requested",
  "version": 1,
  "occurred_at": "2026-09-19T08:00:00Z",
  "payload": {
    "user_id": "531c4d1e-d877-50be-bf27-768458436c9e",
    "to_email": "anton@example.com",
    "template": "EMAIL_VERIFICATION",
    "code": "482913",
    "ttl_minutes": 15
  }
}
```

5.5.2. vacancy.upserted

```
{
  "event_id": "18bb73f7-f54c-5258-8628-6328ea94e1dc",
  "type": "vacancy.upserted",
  "version": 1,
  "occurred_at": "2026-09-19T08:00:00Z",
  "payload": {
    "vacancy_id": "4fc79438-2514-51a8-9796-426af8d8f73d",
    "industry_id": "ef39a9bf-1aa7-521e-870a-46190bc1241a",
    "salary_from": 150000,
    "salary_to": 250000,
    "min_experience_years": 1,
    "is_active": true,
    "skills": [
      {
        "skill_id": "e2ee04fb-d7c7-5d93-8439-208548ade0fa",
        "is_required": true
      }
    ],
    "updated_at": "2026-09-19T08:00:00Z"
  }
}
```

5.5.3. vacancy.deleted

```
{
  "event_id": "2b96536e-0943-5286-8a27-385ca1559b1d",
  "type": "vacancy.deleted",
  "version": 1,
  "occurred_at": "2026-09-19T08:00:00Z",
  "payload": {
    "vacancy_id": "4fc79438-2514-51a8-9796-426af8d8f73d",
    "deleted_at": "2026-09-19T08:05:00Z"
  }
}
```

5.5.4. resume.upserted

```
{
  "event_id": "ab12dfb7-24b2-5258-a4a6-f36083dcaa44",
  "type": "resume.upserted",
  "version": 1,
  "occurred_at": "2026-09-19T08:00:00Z",
  "payload": {
    "resume_id": "06b39995-c3d8-5fcb-831f-7d1fd3421f6e",
    "user_id": "531c4d1e-d877-50be-bf27-768458436c9e",
    "industry_id": "ef39a9bf-1aa7-521e-870a-46190bc1241a",
    "salary_exp_from": 150000,
    "salary_exp_to": 220000,
    "is_published": true,
    "experience_periods": [
      {
        "start_date": "2024-09-01",
        "end_date": null
      }
    ],
    "skill_ids": [
      "e2ee04fb-d7c7-5d93-8439-208548ade0fa",
      "982780ef-4d5a-5e5e-8e6b-8a3eb4d480be"
    ],
    "updated_at": "2026-09-19T08:00:00Z"
  }
}
```

5.5.5. resume.deleted

```
{
  "event_id": "272a0d20-1091-5be5-86c9-c9fd8b54bb66",
  "type": "resume.deleted",
  "version": 1,
  "occurred_at": "2026-09-19T08:00:00Z",
  "payload": {
    "resume_id": "06b39995-c3d8-5fcb-831f-7d1fd3421f6e",
    "deleted_at": "2026-09-19T08:05:00Z"
  }
}
```

Получатель vacancy.upserted и resume.upserted записывает снимок в индекс целиком (upsert по идентификатору), а resume.upserted с is_published = false удаляет резюме из индекса.

6. Ошибки

6.1. Каталог ошибок межсервисного API

HTTP	Код	Сообщение	Когда возникает
401	SERVICE_UNAUTHORIZED	Сервисный токен недействителен	Заголовок Authorization отсутствует, токен просрочен или подпись не проходит проверку
401	INVALID_CLIENT	Неверные учётные данные сервиса	client_id или client_secret не совпадают с зарегистрированными
403	SERVICE_FORBIDDEN	Операция недоступна вызывающему сервису	Токен выдан для другого сервиса (aud) либо клиенту не разрешена операция или параметр
403	AUDIENCE_NOT_ALLOWED	Клиенту не разрешено обращаться к этому сервису	Для пары client_id и audience токен не выдаётся
404	NOT_FOUND	Адрес не найден	Эндпоинт отсутствует или отключён (например, dev/mailbox при MAIL_DEBUG=false)
404	VACANCY_NOT_FOUND	Вакансия не найдена	Вакансия с указанным id не существует
404	RESUME_NOT_FOUND	Резюме не найдено	Резюме с указанным id не существует
404	JOB_NOT_FOUND	Задача не найдена	Задача перестроения индекса с указанным id не существует
409	INDEX_REBUILD_IN_PROGRESS	Перестроение индекса уже выполняется	Для этой области уже запущена задача в состоянии QUEUED или RUNNING
422	VALIDATION_ERROR	Ошибка валидации данных	Параметры или тело запроса не прошли проверку; подробности в details
429	TOO_MANY_REQUESTS	Слишком много запросов	Превышен лимит запросов; повторите позже (заголовок Retry-After)
500	INTERNAL_ERROR	Внутренняя ошибка сервера	Непредвиденная ошибка на стороне сервиса
503	SERVICE_UNAVAILABLE	Сервис временно недоступен	Сервис или его зависимость (БД, брокер, другой сервис) недоступны либо открыт circuit breaker; см. Retry-After
504	UPSTREAM_TIMEOUT	Превышено время ожидания зависимого сервиса	Вызов зависимого сервиса не уложился в таймаут

Уже есть в каталоге ДЗ 2: VACANCY_NOT_FOUND, RESUME_NOT_FOUND, VALIDATION_ERROR, TOO_MANY_REQUESTS, INTERNAL_ERROR; код NOT_FOUND приложение уже использует для несуществующих адресов. Новые для внутреннего API: SERVICE_UNAUTHORIZED, INVALID_CLIENT, SERVICE_FORBIDDEN, AUDIENCE_NOT_ALLOWED, JOB_NOT_FOUND, INDEX_REBUILD_IN_PROGRESS, а SERVICE_UNAVAILABLE и UPSTREAM_TIMEOUT становятся общими для обоих API.

6.2. Как ошибки зависимостей превращаются в ответ клиенту

Ситуация	Что получает клиент	Что происходит внутри
Зависимый сервис не отвечает (соединение отклонено, открыт circuit breaker)	503 SERVICE_UNAVAILABLE с заголовком Retry-After	Операция не выполняется, данные не меняются; для удаления вакансии возвращается прежний is_active
Зависимый сервис не ответил за отведённое время	504 UPSTREAM_TIMEOUT	Идемпотентные запросы повторяются до двух раз, затем ошибка
Вакансия не найдена в Vacancy Service при отклике	404 VACANCY_NOT_FOUND, как в ДЗ 2	Ответ 404 зависимого сервиса переводится в ошибку публичного API
Резюме из отклика не найдено в Resume Service	422 VALIDATION_ERROR (поле resume_id), как в ДЗ 2	То же
Сервисный токен отклонён (SERVICE_UNAUTHORIZED, SERVICE_FORBIDDEN)	500 INTERNAL_ERROR	Ошибка конфигурации, а не клиента: подробности пишутся в журнал с request_id, срабатывает оповещение
Недоступен брокер сообщений	Ответ не меняется	Событие остаётся в outbox_events и публикуется после восстановления; индекс и письма запаздывают
Недоступен только Reference или Identity при чтении	Обычный ответ с пустыми названиями или именами	Используется кеш или пустые значения (таблица в разделе 3.3); ошибка записывается в журнал

6.3. Дополнение к публичному каталогу ошибок

В публичный каталог из ДЗ 2 добавляются два кода, которые появляются только из-за сетевых вызовов между сервисами: 503 SERVICE_UNAVAILABLE («Сервис временно недоступен», заголовок Retry-After) и 504 UPSTREAM_TIMEOUT («Превышено время ожидания зависимого сервиса»). Остальные коды и HTTP-статусы публичного API не меняются.

7. Требования

7.1. Функциональные требования к микросервисной архитектуре

ID	Требование
MS-FR-1	Публичное API (50 операций из ДЗ 2), коды ответов и формат ошибок сохраняются без изменений; клиенты обращаются только к API Gateway
MS-FR-2	Каждый сервис владеет своей БД; доступ к чужим данным возможен только через внутреннее API или события
MS-FR-3	Шлюз маршрутизирует запросы по таблице раздела 2.3, проверяет JWT, применяет лимит запросов и не публикует /internal/**
MS-FR-4	Identity подписывает JWT ключом RS256 и публикует ключи через JWKS; шлюз и сервисы проверяют подпись самостоятельно
MS-FR-5	Внутренние вызовы аутентифицируются сервисными токенами с ограничением по получателю (aud); контакты пользователей выдаются только Application Service
MS-FR-6	Правила RESTRICT между сервисами (вакансия и резюме с откликами) реализуются через applications/exists и схему «закреть, затем проверить»
MS-FR-7	Ссылки на отрасли и навыки проверяются через Reference Service при каждой записи; справочники не удаляются
MS-FR-8	Ответы, собранные из данных нескольких сервисов (отклики), формируются владельцем агрегата (Application) пакетными запросами без обращения по одному объекту
MS-FR-9	События публикуются через outbox, потребители идемпотентны (inbox), формат сообщений версионизируется
MS-FR-10	Recommendation строит индекс из событий и умеет полностью перестроить его из выгрузок Vacancy и Resume
MS-FR-11	Письма с кодами отправляет Notification Service асинхронно с повторами; в тестовом стенде код доступен через /dev/mailbox
MS-FR-12	Все сервисы возвращают ошибки в едином формате и передают сквозной request_id
MS-FR-13	При недоступности зависимости сервис отвечает 503 или 504 и не оставляет данные в промежуточном состоянии

7.2. Нефункциональные требования

ID	Область	Требование
MS-NFR-1	Производительность	95 % запросов публичного API отрабатывают быстрее 500 мс при нормальной нагрузке; тайм-аут внутреннего вызова 2 с (пакетного 3 с)
MS-NFR-2	Доступность	Отказ Recommendation или Notification не влияет на остальные операции; отказ Reference не блокирует чтение (работает кеш)
MS-NFR-3	Устойчивость	Повторы не более двух и только для безопасных запросов; circuit breaker открывается после 5 ошибок за 10 секунд на 30 секунд
MS-NFR-4	Согласованность	Внутри сервиса строгая (транзакции БД); между сервисами в конечном итоге; отставание индекса рекомендаций не более 5 секунд
MS-NFR-5	Безопасность	Секреты вне кода; TLS между компонентами; у каждого сервиса отдельный пользователь БД; коды не попадают в журналы
MS-NFR-6	Наблюдаемость	Сквозной request_id и transparent; структурные журналы; метрики запросов, ошибок и задержек; health и readiness у каждого сервиса
MS-NFR-7	Масштабируемость	Сервисы не хранят состояние в памяти, кроме кешей, и масштабируются горизонтально; счётчики лимитов лежат в Redis
MS-NFR-8	Развёртывание	Каждый сервис собирается и выпускается независимо в своём контейнере; схемы БД меняются миграциями, synchronize: true отключён
MS-NFR-9	Тестируемость	Контрактные тесты по openapi-internal.yaml; сценарий Postman из ДЗ 3 выполняется после каждого шага миграции без изменений
MS-NFR-10	Переносимость данных	Перенос таблиц без потерь: число строк и контрольные суммы совпадают до и после; откат возможен до конца периода наблюдения

8. План перехода от монолита

8.1. Стратегия

Разделение выполняется постепенно (по схеме «душитель», *strangler fig*): шлюз ставится перед монолитом, затем сервисы вырезаются по одному, а монолит уменьшается, пока не исчезнет. Порядок идёт от сервисов, которые почти ни от кого не зависят, к тем, которые зависят от многих: Notification, Reference, Identity, Recommendation, Resume, Vacancy и в конце Application. На каждом шаге публичное API остаётся неизменным, а сценарий Postman из ДЗ 3 (40 запросов, 202 теста) должен проходить полностью. Пока часть функций остаётся в монолите, он временно реализует нужные внутренние эндпоинты (например, `applications/exists`), чтобы новые сервисы уже работали по итоговому контракту.

8.2. Шаги

Шаг	Что делаем	Готово, когда	Откат
0. Подготовка	Разложить код монолита по доменным модулям и запретить обращения к чужим таблицам; заменить <code>synchronize: true</code> миграциями TypeORM; ввести общий шаблон сервиса (<code>health</code> , <code>request_id</code> , формат ошибок); поднять RabbitMQ и Redis; настроить CI	Приложение работает как раньше; в коде нет чужих репозиторий; Postman проходит	Не требуется
1. API Gateway	Поставить шлюз перед монолитом; перенести в него проверку JWT и лимит запросов; закрыть <code>/internal/**</code>	Все 50 операций работают через шлюз; Postman проходит	Вернуть маршрут клиентов на монолит
2. Notification	Монолит публикует <code>identity.email_requested</code> через <code>outbox</code> , отправку выполняет Notification; тестовый ящик переезжает в него	Коды приходят; запросы 0.2 и 1.3 Postman читают код через шлюз	Флагом вернуть отправку внутри монолита
3. Reference	Перенести <code>industries</code> и <code>skills</code> в <code>reference_db</code> ; остальные читают через <code>lookup</code> с кешем; удалить 5 внешних ключей на справочники	№12–14 обслуживает Reference; число строк совпадает; ключи удалены	Вернуть чтение из старых таблиц (данные ещё не удалены)
4. Identity	Перенести <code>users</code> , профили и токены в <code>identity_db</code> ; перейти на RS256 и JWKS; включить сервисные токены; превратить 3 внешних ключа на <code>users</code> в логические	№1–11 работают; пользователи вошли заново (старые токены недействительны)	Вернуть встроенную аутентификацию, пока не закончилось окно перехода
5. Recommendation	Создать индекс, загрузить его через <code>export</code> и <code>rebuild</code> , подписаться на события; сначала сравнить результаты с монолитом, клиенту отдавать старые; затем переключить	<code>match_score</code> совпадает на контрольной выборке; №48–49 работают	Вернуть маршрут на реализацию в монолите
6. Resume	Перенести таблицы резюме в <code>resume_db</code> ; обращения к откликам через <code>applications/exists</code> и <code>access-check</code> ; публиковать <code>resume.*</code>	№15–26 работают; Postman проходит	Вернуть таблицы и маршрут
7. Vacancy	Перенести компании, вакансии, навыки вакансий и избранное в <code>vacancy_db</code> ; реализовать закрытие перед удалением; публиковать <code>vacancy.*</code>	№27–38 и №45–47 работают; Postman проходит	Вернуть таблицы и маршрут
8. Application	Перенести <code>applications</code> в <code>application_db</code> ; добавить и заполнить <code>applicant_user_id</code> и <code>employer_user_id</code> из <code>resumes</code> и <code>companies</code> ; включить сборку ответов и сверку целостности; выключить монолит	№39–44 работают; монолит остановлен	До остановки монолита вернуть таблицу и маршрут

8.3. Перенос данных на каждом шаге

- Создать БД сервиса и схему миграциями; выдать сервису отдельного пользователя.
- Скопировать таблицы (`pg_dump` или `COPY`) в новую БД.

- Догонять изменения до момента переключения: логическая репликация PostgreSQL или запись в обе БД на время окна.
- Сверить данные: число строк по таблицам, контрольные суммы по ключевым столбцам, выборочное сравнение ответов API.
- Переключить маршрут шлюза на новый сервис: сначала чтение, затем запись; старые таблицы оставить только для чтения.
- После периода наблюдения (например, две недели) удалить старые таблицы и внешние ключи.

8.4. Проверка результата

- Регрессия: сценарий Postman из ДЗ 3 после каждого шага; он проверяет основной процесс от регистрации до обработки отклика и негативные случаи.
- Контрактные тесты внутренних вызовов по openapi-internal.yaml.
- Проверка отказов: остановка каждого зависимого сервиса и сравнение поведения с таблицами разделов 3.3 и 6.2.
- Сверка данных и сравнение ответов старой и новой реализаций (для рекомендаций и списков).

8.5. Риски

Риск	Мера
Рост задержек из-за цепочек вызовов	Пакетные запросы вместо N+1, кеш справочников, параллельные вызовы, тайм-ауты; цель 95 % ответов быстрее 500 мс
Расхождение данных между сервисами	Outbox и inbox, сверка целостности, перестроение индекса, единственный владелец каждой записи
Взаимные вызовы Application, Vacancy и Resume	Вызовы не вложены (глубина не более 1), у каждого тайм-аут; при росте связности выделить общий контракт
Отказ зависимости каскадом	Circuit breaker, изолированные пулы соединений, деградация некритичных данных
Код подтверждения виден в брокере	TLS, ограниченное время жизни сообщений, доступ по учётной записи сервиса, код не пишется в журналы
Потеря сессий при смене алгоритма JWT	Запланированное окно, предупреждение пользователей; refresh-токены переносятся, вход выполняется заново
Сложность эксплуатации восьми компонентов	Единый шаблон сервиса, контейнеры и оркестрация, централизованные журналы, health и readiness
Сложная отладка	Сквозные request_id и traceparent, одинаковый формат ошибок и журналов

8.6. Границы работы

В работе проектируется целевая архитектура и путь к ней. Не выполняются: реализация кода сервисов, настройка оркестратора и мониторинга, нагрузочное тестирование. Публичное API и бизнес-правила из ДЗ 1 и ДЗ 2 не изменяются.

9. Вывод

Монолитное приложение разделено на семь микросервисов и шлюз с соблюдением принципа database-per-service: 15 таблиц распределены по пяти БД (Identity, Reference, Vacancy, Resume, Application), ещё две БД (Recommendation и Notification) содержат только новые данные. Из 19 внешних ключей монолита 9 остаются внутри своих БД, а 10

заменены логическими ссылками с проверками через внутреннее API и схемой «сначала закрыть, потом проверить». Межсервисное API описано в формате OpenAPI 3.0 (17 операций, 44 схемы, единый каталог ошибок), события описаны отдельными контрактами. Сформированы функциональные и нефункциональные требования и пошаговый план перехода с проверкой сценарием Postman на каждом шаге. К отчёту приложены файлы openapi-internal.yaml и microservices.drawio.